

45. Un lutin, au pixel près

Ce qui sépare un jeu d'action d'un jeu de cases.

1. Ce qu'il faut comprendre

Le décor vit sur une grille de huit pixels. Un personnage, non : s'il ne pouvait s'arrêter que sur des multiples de huit, il ne sauterait pas, il **se téléporterait**.

`sprite(numero, x, y, tuile)` pose un carré de huit sur huit **où l'on veut**. Le matériel en a quarante, numérotés de 0 à 39.

Il n'y a rien à faire pour l'effacer : on le repose ailleurs à chaque image, et il a bougé.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **651 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          57 o ← la console, à l'allumage - la mise en route de la console
main()             137 o ← démarrage - ta fonction
VBlank              8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage       28 o ← image() - attend l'image suivante
AttendreVBlank      16 o ← démarrage - attend que l'écran soit libre
AttendreAcces       12 o ← EcrireTexte - attend de pouvoir écrire une case
EcrireTexte         10 o ← texte() - écrit un texte, lettre par lettre
RangerLutins        11 o ← démarrage - range les 40 lutins hors de l'écran
InstallerTransfert  15 o ← démarrage - installe la copie des lutins en mémoire rapide
CopierTuiles        103 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte        14 o ← démarrage - vide les deux cartes de l'écran
LireManette         37 o ← bouton() - lit les huit boutons
Tuiles              176 o ← CopierTuiles - 10 lettres sur 43, et 1 tuile dessinée
DonneesTransfert    10 o ← InstallerTransfert - la petite routine qui copie les lutins
"CROIX POUR BOUG..." 17 o ← texte() - un texte gravé
```

Lettres gravées : BCEGIOPRUX

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

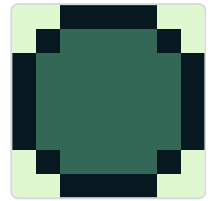
```
sprite() × 1      23 o sur place, dans main()
bouton() × 4      20 o sur place, dans main() + LireManette (37 o)
texte() × 1       11 o sur place, dans main() + EcrireTexte (10 o)
image() × 1        6 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <Tuile>    // un dessin de 8 × 8 pixels
#include <texte>     // écrit un texte à l'écran
#include <bouton>    // lit un bouton de la manette
#include <sprite>    // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — BALLE

```
Tuile BALLE = {
    "00333300",
    "03222230",
    "32222223",
    "32222223",
    "32222223",
    "32222223",
    "32222223",
    "03222230",
    "00333300",
};
```



à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    uint8_t x = 20;
    uint8_t y = 60;

    texte(3, 2, "CROIX POUR BOUGER");

    while (true) {
        image();

        if (bouton(DROITE) && x < 150) x += 2;
        if (bouton(GAUCHE) && x > 2) x -= 2;
        if (bouton(BAS) && y < 135) y += 2;
        if (bouton(HAUT) && y > 2) y -= 2;

        sprite(0, x, y, BALLE);
    }

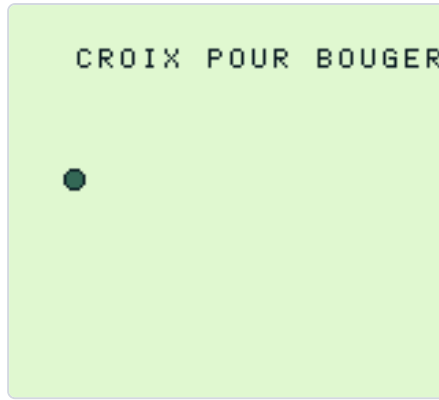
    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



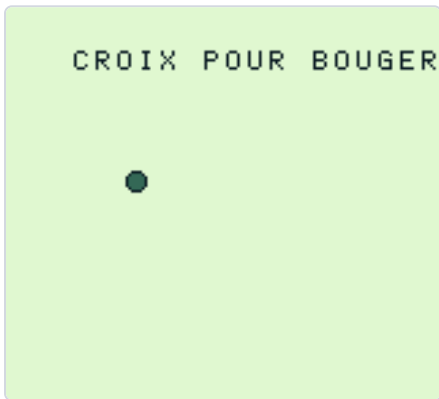
la toute première image — la console vient de démarrer — image 1



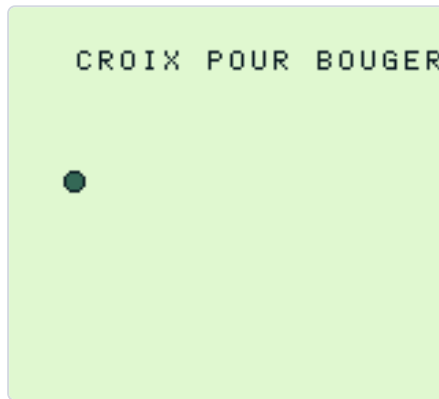
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

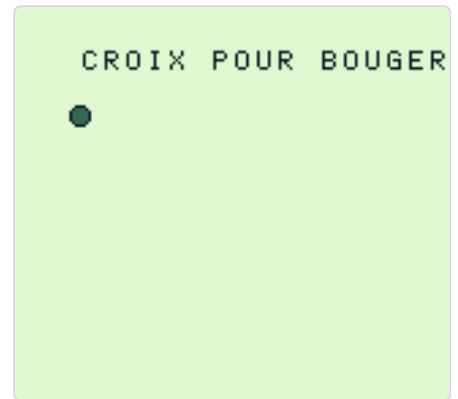
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



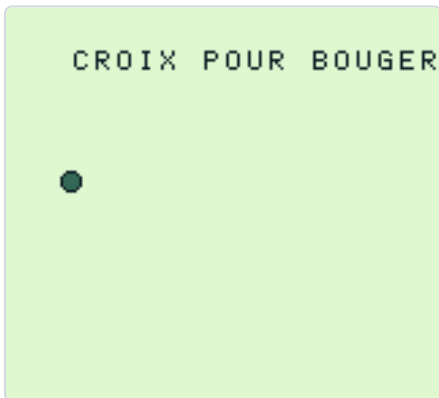
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Une boule qu'on déplace à la croix, sans à-coups.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le lutin est à l'écran
- ✓ il porte la tuile dessinée
- ✓ DROITE le déplace (20 vers 44)
- ✓ BAS aussi
- ✓ il se pose HORS de la grille de huit (positions prises : 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42)

Ensuite : **46. Un objet qui tombe** — chapitre 7 : Le mouvement. · gameboy3 — écrire une cartouche Game Boy en C++